QuantumEnergyOS — ISO Construction Guide

Complete Technical Blueprint for Compiling, Packaging, and Generating ISO Images

Document Ver: 2.0.3 **Target OS:** Linux x86_64 **Arch Profile:** ArchISO / LiveCD **Author:** OS Build Team

1. Executive Summary & Overview

QuantumEnergyOS is an advanced Linux-based operating system designed for hybrid quantum-photonic energy orchestration, climate grid balancing, and real-time power optimization. This guide details the complete build workflow to synthesize a bootable ISO image (LiveCD/LiveUSB) incorporating the core Rust photonic drivers, Python climate orchestrator engines, and web monitoring interfaces.

> **Important:** Building this ISO requires administrative (root or sudo) privileges on an Arch Linux or Arch-derived build host due to namespace chroot environments used by `mkarchiso`.

2. System Requirements & Dependencies

Before initializing the build pipeline, ensure your host environment meets the necessary hardware and software constraints:

RESOURCE / TOOL	MINIMUM REQUIRED	RECOMMENDED SPECIFICATION
Host Architecture	x86_64 with Virtualization (VT-x/AMD-V)	Multi-core x86_64 CPU (8+ cores)
RAM	8 GB RAM	16 GB+ RAM (Saves build time in RAM-disk)
Free Disk Space	25 GB Available Space	50 GB NVMe Storage
Core Build Tools	`archiso`, `mkarchiso`, `git`	`archiso`, `rustup`, `python-pip`, `docker`
Languages & Runtimes	Rust 1.70+, Python 3.10+, Node.js 18+	Rust Stable Toolchain, Python 3.11, Node 20 LTS

3. Repository Directory Structure

The ISO generation process depends on key source directories and configuration templates present in the main repository:

```
QuantumEnergyOS-V.03-main/
├── AppData/Local/Programs/Microsoft VS Code/QuantumEnergyOS/
│   ├── buildsystem/
│   │   └── build.ps1              # PowerShell automation script for Windows/cross-builds
│   ├── climate_orchestrator/      # Python climate bridge & quantum models
│   ├── livecd/
│   │   └── profiledef/
│   │       └── packages.x86_64    # Target package list for the ISO rootfs
│   ├── photonic-bridge/           # Rust bindings for optical hardware interconnects
│   ├── photonic-core/             # Rust optical optimizer library
│   ├── web-dashboard/             # React/TypeScript monitoring frontend
│   ├── Dockerfile                 # Containerized build environment definition
│   └── Makefile                   # Unified build automation pipeline
├── Build-ISO.ps1                  # Root PowerShell build utility
├── Makefile                       # Native build engine entrypoint
└── GUIA-CONSTRUCCION-ISO.pdf      # Artifact baseline document
```

4. Core Modules Pre-Compilation

Before embedding custom binaries into the ISO profile filesystem (`airootfs`), you must compile the native components (Photonic Core, Photonic Bridge, and Web Dashboard).

4.1 Compiling Photonic Core & Bridge (Rust)

Compile the optimized binaries for the Linux target architecture:

```
# Navigate to Photonic Core and compile release target
cd photonic-core
cargo build --release

# Navigate to Photonic Bridge and compile release target
cd ../photonic-bridge
cargo build --release
```

4.2 Building the Web Dashboard (Node / React)

Bundle the TypeScript frontend into static production assets:

```
cd ../web-dashboard
npm install
npm run build
```

5. ISO Profile Configuration

The ISO image is generated from the configuration files inside `livecd/profiledef/`.

5.1 Managing ISO Packages (packages.x86_64)

Ensure essential system packages, Python runtimes, and optical drivers are declared in `livecd/profiledef/packages.x86_64`:

```
# Base System
linux
linux-firmware
base
base-devel

# Networking & Utilities
dhcpcd
iwd
networkmanager
openssh
curl
git

# QuantumEnergyOS Dependencies
python
python-pip
python-numpy
python-scipy
rust
hwloc
opencl-icd-loader
```

5.2 Injecting Custom Binaries & Services

Copy compiled binaries into the live filesystem template structure before launching the ISO build engine:

```
# Prepare custom directory structure
mkdir -p livecd/airootfs/usr/local/bin
mkdir -p livecd/airootfs/var/www/quantum-dashboard

# Copy Rust Binaries
cp photonic-core/target/release/photonic-core livecd/airootfs/usr/local/bin/
cp photonic-bridge/target/release/photonic-bridge livecd/airootfs/usr/local/bin/

# Copy Frontend Assets
cp -r web-dashboard/build/* livecd/airootfs/var/www/quantum-dashboard/
```

6. Executing the ISO Build Pipeline

Option A: Automated Makefile Workflow (Recommended for Linux)

Use the root Makefile to run clean preparation, binary compilation, and ISO generation sequentially:

```
# Clean existing build workspace
make clean

# Build all sub-services and compile ISO
make iso
```

Option B: Manual ArchISO Command Line Execution

If executing manually on an Arch host, run `mkarchiso` directly:

```
# Create output directory for completed ISO
mkdir -p out/

# Build ISO using the specified profile
sudo mkarchiso -v -w work/ -o out/ livecd/profiledef/
```

> **Success Output:** Upon completion, your bootable image will be available at: `out/QuantumEnergyOS-v0.03-x86_64.iso`

Option C: Containerized Build via Docker

To avoid host system contamination or build on non-Arch hosts, leverage the repository Dockerfile:

```
# Build Docker image containing the build environment
docker build -t quantum-iso-builder -f docker/Dockerfile .

# Run container in privileged mode to generate ISO
docker run --privileged -v $(pwd)/out:/build/out quantum-iso-builder
```

7. ISO Verification & Testing

Prior to flashing onto physical installation media, verify the ISO image integrity and boot performance in QEMU:

```
# QEMU Test Command with KVM Acceleration
qemu-system-x86_64 -enable-kvm -m 4096 -smp 4 -cdrom out/QuantumEnergyOS-v0.03-x86_64.iso -boot d -vga virtio
```

8. Troubleshooting & Common Failures

SYMPTOM / ERROR	ROOT CAUSE	RESOLUTION STEP
`pacman lock file error`	Interrupted build left a stale lock file in chroot.	Run `sudo rm -f work/x86_64/airootfs/var/lib/pacman/db.lck` and restart.
`No space left on device`	`/tmp` or work dir ran out of allocated space.	Change work directory to a partition with 30GB+ space using `-w /path/to/work`.
Missing dynamic library (e.g. `libopencl.so`)	Dependency omitted from `packages.x86_64`.	Add missing library package name to `packages.x86_64` and rebuild.
Rust target triple failure	Mismatch between host cross-compilation target and guest ISO.	Ensure target is set explicitly: `cargo build --target x86_64-unknown-linux-gnu --release`.

> **Security & Deployment Note:** Ensure all default cryptographic keys, API tokens, and temporary certificates configured in `.env.example` are replaced or purged prior to distributing production ISO images.